TD2
DPI, modèle Python

Thomas Sarno

Septembre 2026

1 Introduction

Il vous est demandé de rédiger un compte-rendu pour ce TD dans lequel vous ferez figurer toutes les remarques pertinentes qui vous semblent nécessaires à la bonne compréhension du problème posé et de la solution apportée.
Bien évidement il s'agit d'un travail personnel.
Dans le TD2 nous allons rendre l'environnement autonome. La génération de vecteurs d'entrées se fera directement depuis le SystemVerilog tandis que la vérification s'appuiera sur l'utilisation d'un modèle python.
Ce TD sera l'occasion de mieux appréhender "`Direct Programming Interface`" qui permet d'interfacer SystemVerilog avec d'autres langages comme par exemple le langage C/C++. Mais aussi d'aller plus loin en exploitant la facilité d'utilisation du langage Python.

Ci dessous, pour rappel, vous trouverez un diagramme bloc (Figure **??**) ainsi qu'un chronogramme (Figure **??**) décrivant le fonctionnement de cet AES.

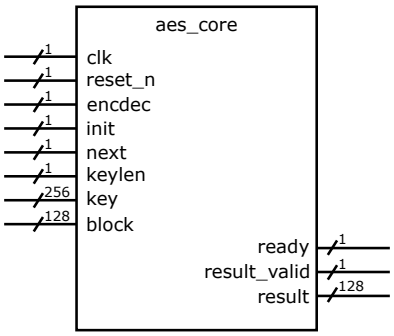


FIGURE 1 – aes_core

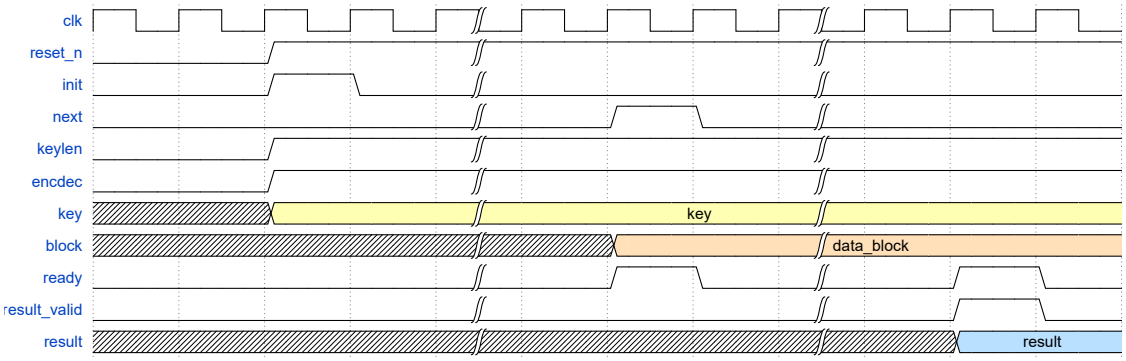


FIGURE 2 – Chronogramme de l'aes_core

2 Démarrage du TD2

Pour démarrer le TD2, vous devez récupérer l'archive du TD2 sur votre machine. Le testbench présent dans l'archive est vide, vous devrez utiliser ce que vous avez fait lors des séances précédentes et y ajouter les nouvelles fonctionnalités demandées dans ce TD.

La hierarchie des fichiers est la suivante :

- `TP` : contient les fichiers sources du testbench, le script python et le code C pour l'interface avec le testbench.
- `TD` : contient ce fichier .pdf du compte rendu.
- `rtl` : contient les fichiers sources du circuit matériel.
- `build.sh` : script de compilation et de lancement de la simulation.
- `build_c.sh` : script de compilation du code C pour l'interface avec le testbench.

Conservez cette hierarchie de fichiers pour le bon fonctionnement. En effet les scripts de compilation et de lancement de la simulation sont écrits pour fonctionner avec cette hierarchie. De même d'un TD à l'autre si vous voulez avoir un dossier different par TD, gardez cette hierarchie pour chacun d'eux et copiez les fichiers nécessaires dans le nouveau dossier.

3 Interface DPI-C

Dans un premier temps vous allez créer un fichier .c avec une seule fonction très simple qui aura pour entrée un entier et pour sortie l'entrée multipliée par 2.

Vous compilerez ce fichier avec la commande :

```
gcc -g -DDPI -DPYTHON_SHARED_LIB="$PYTHON_SHARED_LIB" -I . -I $MGLS_INCLUDE
-I $PYTHON_INCLUDE/python${PYTHON_VERSION}/ -shared -Bsymbolic -fpic TP/call_python.c
$PYTHON_SHARED_LIB $PYTHON_LIB/libpython${PYTHON_VERSION}.so.1.0 -o TP/call_python.so
```

La variable `$MGLS` pointe vers le dossier *include* de l'emplacement de modelsim et les variables `$PYTHON_LIB`, `$PYTHON_SHARED_LIB` pointent respectivement vers les bibliothèques Python et la bibliothèque partagée Python. La variable `$PYTHON_INCLUDE` quand à elle pointe vers le dossier *include* de l'installation de Python. Prenez garde à bien placer le fichier objet dans le dossier duquel vous lancez votre simulation.
Cette commande est détaillée dans le script `build_c.sh`, mais vous pouvez l'exécuter manuellement si nécessaire à condition de bien initialiser les variables.

Il faut ensuite connecter cette fonction au testbench systemVerilog. Comme indiqué dans le cours, vous utiliserez la commande import dans l'en-tête de votre testbench (en remplaçant le nom et les types des entrées/sorties) :

```
import "DPI-C" function byte cfunction(input byte a)
```

Et finalement, lancer la fonction depuis le bloc `initial` de votre testbench.

Lors du lancement de la simulation il faut indiquer à l'outil que vous voulez utiliser une bibliothèque externe qui se connecte à votre testbench via DPI. Cela se fait avec l'option *-sv_lib c_library* quand le nom de la bibliothèque est *c_library.so*.

4 Utilitaire de vérification Python

A partir des même commandes que celle du script du TD1 qui a permis de générer les fichiers d'entrée et de sortie de référence de l'AES, il vous faudra écrire un nouveau script qui sera appelé par le socket server (lui aussi en python) contenant une fonction pour calculer un chiffrement 128 bit ECB de l'AES.

```
def AES_ref_py(pt_sv, key_sv):
  # your_code
  return ct_py
```

Testez et vérifiez votre script avec les vecteurs de test du FIPS197 (`https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.197.pdf`) :

— `key=0x2b7e151628aed2a6abf7158809cf4f3c`
— `text_in=0x3243f6a8885a308d313198a2e0370734`
— `text_out=0x3925841d02dc09fbdc118597196a0b32`

5 Mise en place de pythonC

Par la suite le langage C sera utilisé uniquement pour s'interfacer entre le testbench en systemVerilog et le script python. Pour cela nous utilisons la librairie `Python.h` qui permet d'appeler des fonctions python depuis le langage C.

Il vous faudra donc utiliser le fichier .c qui contiendra les fonctions d'initialisation et de finalisation de l'environnement python ainsi qu'une fonction qui permettra d'appeler la fonction `AES_ref_py` du script python.

L'appel du script python se fait en 3 étapes :
— Initialisation de l'environnement python avec la fonction `python_initialize_c`
— Appel du module python, qui est faite par la fonction `aes_c` qui prend en entrée deux chaînes de caractères (texte clair et clé) et renvoie une chaîne de caractères (texte chiffré). Si le module python est modifié pour avoir des entrées/sorties différentes, il faudra modifier la fonction `aes_c` pour qu'elle corresponde à la nouvelle interface.
— Finalisation de l'environnement python avec la fonction `python_finalize_c`.
En l'état, la fonction `aes_c` est appelée depuis le testbench avec des chaînes de caractères. Il faudra donc convertir les entrées/sorties du testbench en chaînes de caractères avant l'appel de la fonction `aes_c` et convertir la sortie de la fonction `aes_c` en entier pour pouvoir comparer avec le résultat du circuit matériel.

6 Connexion au testbench systemVerilog

La fonction `aes_c` peut maintenant être utilisée dans le testbench, elle permet d'envoyer des arguments au script python et de recevoir sa réponse.
N'oubliez pas les fonctions d'initialisation et de fermeture, la fonction `python_initialize_c` doit être appelée dans le bloc `initial` du testbench avant tout appel à une fonction python tandis que la fonction `python_finalize_c` doit être appelée à la fin du testbench, avant le mot-clé $finish.

Vous pouvez soit faire la comparaison avec le résultat de votre hardware directement en python, soit renvoyer le résultat du chiffrement, réceptionner le résultat du python et le comparer dans le testbench system verilog.

7 Génération aléatoire de textes clairs et clés de chiffrement

7.1 Vérification aléatoire

Maintenant que la vérification fonctionne pour une paire de texte clair/clé de référence, pour faire une véritable vérification il faut pouvoir tester sur un nombre quelconque de paires aléatoires texte clair/clé.

Commencez par créer une classe `Input_class` dans laquelle vous déclarez deux *logic array* de 128 bits avec le mot clé `rand`, un pour le texte claire et un pour la clé.

Dans une boucle générez de nouveaux couples texte clair/clé et envoyez-les au python et au circuit matériel.

Définissez le nombre d'itération de la boucle avec un paramètre dans la commande de simulation en utilisant `+ITERATION_NB=xxx`. Vous pouvez récupérer cette valeur dans le testbench avec la fonction :

```
valid = $value$plusargs("ITERATION_NB=%d",iteration_nb_s);
```